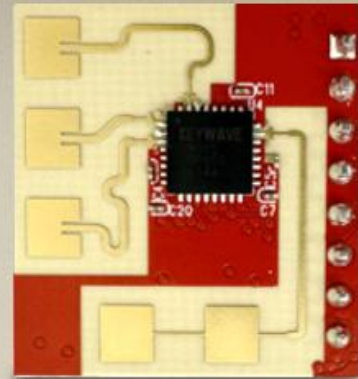


KEYWAVE

Keywave - KW307

Human Presence Sensor Module

- Revolutionary new Radar technology
- Ultra-low power consumption
- Detects multiple and stationary People
- Reduces false alarms



SDK User Guide

Version 2.5.1 | 2026-08-06

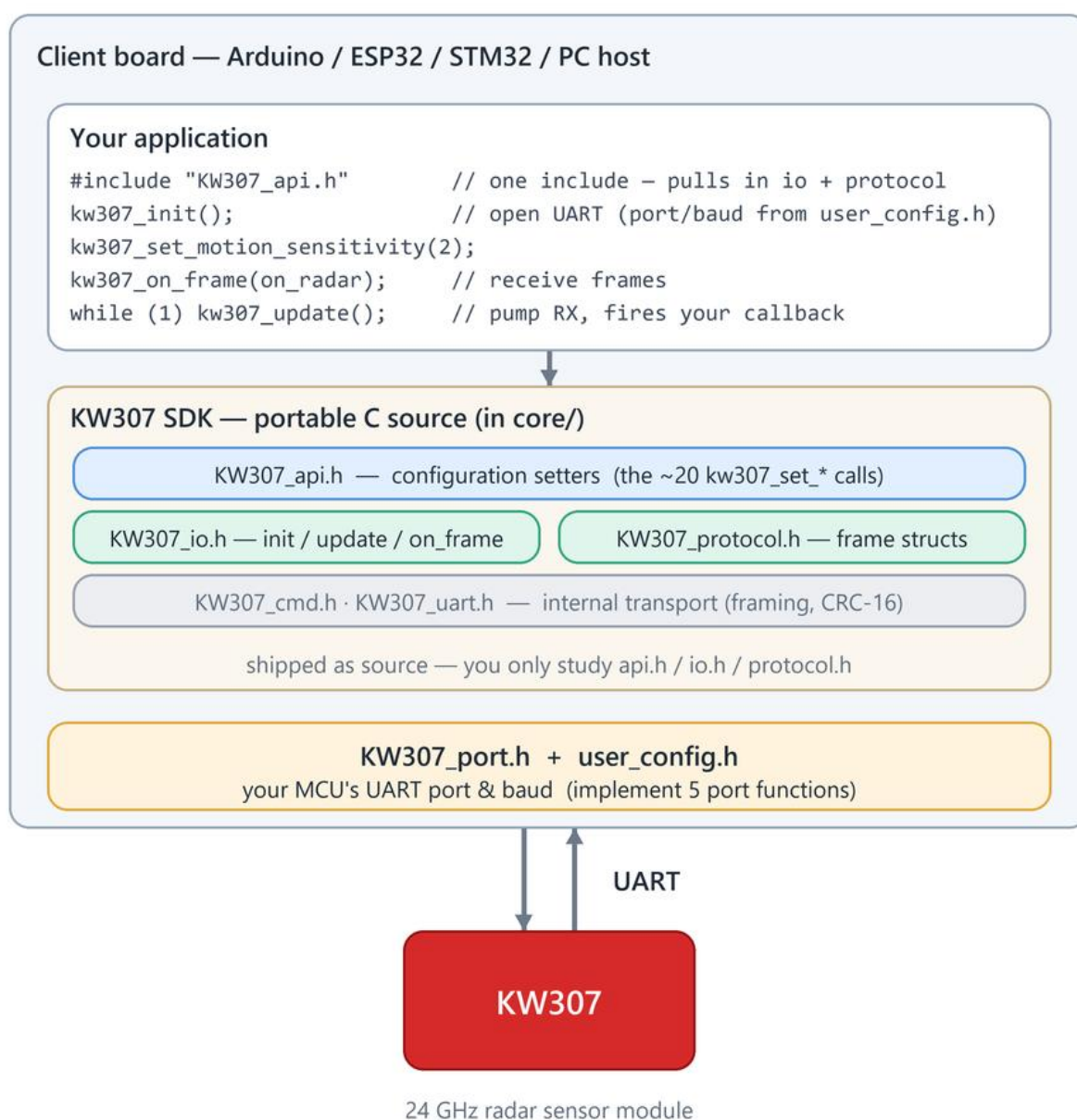
KW307: Human Presence Sensor with Ultra-Low Power Module

Overview

24 GHz human-presence radar · ultra-low power · UART interface

The **KW307** is a compact 24 GHz RF human-presence sensor module that detects both moving and stationary people, indoors and outdoors. It reports per-target distance and angle over UART, with configurable range, field of view, gain, sensitivity and hold time, and tracks up to three targets (two moving + one stationary) in the default output frame.

KW307 SDK — how it fits together



Key Features

- ◆ 24 GHz RF-based human presence sensing.
- ◆ Detects both moving and stationary people.
- ◆ Multi-target tracking (2 moving + 1 stationary in the default frame).
- ◆ Per-target distance and angle reported over UART.
- ◆ Configurable detection range, field of view, gain, sensitivity, hold time.

1. UART Protocol

1.1 Physical Layer

Configure users host UART port with the following settings:

UART Configuration	Description
Baud Rate	115200 bps
Data Bits	8
Parity	None
Stop Bits	1
Flow Control	None
Logic Level	3.3V TTL (Low: 0V, High: 3.3V)
Byte Order	Big-Endian (MSB first)

1.2 Frame Format

Every frame — whether a host command, a read request, an ACK response, or a periodic output follows this structure.

AA 55(W)/56(R) [CMD] [LEN_H] [LEN_L] [DATA...] [CRC16_H] [CRC16_L]

Field	Length (Bytes)	Description
SOF [0]	1	Fixed: 0xAA
SOF [1]	1	0x55 = Write frame / 0x56 = Read frame
CMD	1	Command ID (see Command Table)
LEN	2	Byte count of DATA only, Big-Endian
DATA	0~N	Parameter payload
CRC16	2	CRC-16/CMS (non-reflected), Poly = 0x8005, Seed=0xFFFF, Scope = CMD+LEN_H+LEN_L+DATA (Big-Endian output)

Note: Start of Frame (SOF) bytes (AA 55/56) are not included in the CRC-16 calculation.

1.3 MCU Wake-Up Sequence

The module enters low-power mode when idle. Before sending every command, transmit at least 5 wake-up bytes (0x00) immediately followed by the command frame with no gap.

00 00 00 00 00 AA 55/56 [CMD] [LEN_H] [LEN_L] [DATA...] [CRC16_H] [CRC16_L]

Do not insert any delay between the wake-up bytes and the command.

A gap will cause the MCU to re-enter low-power mode before the command arrives.

The 0x00 (0b00000000) is used as the wake-up byte because its bit pattern produces repeated transitions on the UART RX line, reliably waking the module from low-power mode.

1.4 Write and Read Operations

Read and Write share the same CMD ID. Direction is determined solely by SOF [1].

Write: AA **55** [CMD] [LEN_H] [LEN_L] [DATA...] [CRC16_H] [CRC16_L]

Read request: AA **56** [CMD] 00 00 [CRC16_H] [CRC16_L]

1.4.1 Write Response (ACK)

Success: AA 55 **AC** 00 02 [CMD] **00** [CRC16_H] [CRC16_L]

Failure: AA 55 **AC** 00 02 [CMD] **[ERR]** [CRC16_H] [CRC16_L]

1.4.2 Read Response

A successful read returns two frames in sequence:

Frame 1 (ACK): AA **56** AC 00 02 [CMD] 00 [CRC16_H] [CRC16_L]

Frame 2 (Data): AA **56** [CMD] [LEN_H] [LEN_L] [VALUE...] [CRC16_H] [CRC16_L]

Wait for Frame 1 first.

If STATUS = 0x00, wait for Frame 2. Otherwise handle the error code.

1.5 ACK Status Codes

Value	Status	Description
0x00	OK	Command accepted and executed
0x01	Invalid value	DATA value out of allowed range
0x02	Unknown CMD	CMD ID not recognized
0x03	CRC error	Received CRC-16 does not match calculated value
0x04	Invalid length	LEN does not match expected size for this CMD

1.6 CRC-16 Calculation

Scope: CMD + LEN_H + LEN_L + DATA[0...N-1] (SOF bytes AA 55/56 are excluded).

Algorithm: CRC-16/CMS (non-reflected, MSB-first).

Poly = 0x8005, Seed = 0xFFFF, output = Big-Endian (CRC16_H sent first).

```
uint16_t crc16(const uint8_t *buf, int len) {
    uint16_t crc = 0xFFFF;
    for (int i = 0; i < len; i++) {
        crc ^= (uint16_t)buf[i] << 8;
        for (int b = 0; b < 8; b++) {
            if (crc & 0x8000) crc = (crc << 1) ^ 0x8005;
            else          crc <<= 1;
        }
    }
    return crc; /* CRC16_H = crc >> 8, CRC16_L = crc & 0xFF */
}
```

1.7 Data Encoding Reference

Format	Description	Example
BE int16	2-byte signed integer, Big-Endian (MSB first)	350 -> 01 5E
BE uint16	2-byte unsigned integer, Big-Endian	3000 -> 0B B8
uint8	1-byte unsigned integer	5 -> 05
cm x10 (= mm)	Distance in cm x 10	50.0 cm -> 500 -> 01 F4
deg x10	Angle in degrees x 10	45.0 deg -> 450 -> 01 C2
Q15	Float x 32768, sent as BE int16	0.875 -> 28672 -> 70 00

1.8. Example Frames

1.8.1 Write Examples

Set Power Mode to Normal (CMD 0x10, value=1):

00 00 00 00 00 AA 55 **10** 00 01 **01** 46 24

Set Gain to +9dB (CMD 0x12, value=4):

00 00 00 00 00 AA 55 **12** 00 01 **04** EE 39

Set Detection Range min=500mm(0x01F4) / max=3000mm(0x0BB8) (CMD 0x90, PID 0x0F / 0x10):

Min: 00 00 00 00 00 AA 55 90 00 03 **0F** 01 F4 [CRC]

Max: 00 00 00 00 00 AA 55 90 00 03 **10** 0B B8 [CRC]

Set Detection FOV to 90 deg (CMD 0x90, PID 0x11, value=900 = 0x0384):

00 00 00 00 00 AA 55 90 00 03 11 **03 84** [CRC]

1.8.2 Read Example

Read current Gain (CMD 0x12):

Request: 00 00 00 00 00 AA 56 **12** 00 00 8F 6B

ACK: AA 56 **AC** 00 02 **12** 00 F9 0B

Data: AA 56 **12** 00 01 04 EE 39

2. Radar Detection Output Frame – 27 Bytes

The module periodically sends detection results using CMD 0xF0 (SOF = AA 55).

Output format is selected by CMD 0x11.

Command	Bytes	Field	Type	Description
0xF0	[0-1]	Frame Index	uint16_t	Frame counter, increments every radar frame.
	[2]	Human flag	uint8_t	1 = presence detected, 0 = no presence. (Module's own algorithm output.)
	[3-10] [11-18]	Moving Tracker	KW307Tracker x2	Two moving-target slots; Section 2.1 for field layout
	[19-26]	Stationary Tracker	KW307Tracker	One stationary-target slot; Section 2.1 for field layout.

The first 2 bytes of every output frame are a Frame Index (Big-Endian) for missed-frame detection.

➤ Human_flag

Presence(human)_flag is implemented in module hardware for reference. Customers can re-define their own flag based on detected object result. The Human_flag can adjust by kw307_set_flag_hold_time to prevent the glitch of flag.

2.1 Output Frame Tracker - 8 bytes

Bytes	Field	Type	Description
[0-1]	Distance_mm	uint16_t	Distance in mm (2000mm = 200cm)
[2-3]	Angle_tenth	int16_t	Angle in 0.1-degree units (350 = 35.0 deg)
[4-5]	Magnitude	uint16_t	Signal magnitude
[6]	State	uint8_t	Simple (Default) Moving State: 0=Empty 1=Valid Stationary State: 0=Empty 1=Valid
			Advanced Moving State: 0=Empty 1=Checking 2=Birth 3=Valid Stationary State: 0=Empty 1=Valid 2=Hidden
[7]	Counter	uint8_t	Internal age

Tracker **state** and **counter**:

Each target slot carries a “State” — whether the slot holds a real target — and a “Counter” — how confident and how fresh it is.

➤ State

The values depend on `kw307\_set\_tracker\_state\_advanced\_mode()`; the simple form is the default.

Use a slot only when it is “Valid” — “state == 1” in the simple form, “moving[].state == 3” and “stationary[].state == 1” in the advanced form. Test for Valid, not “state != 0”.

The simple form zeroes every field of a non-Valid slot, so there is nothing to read from it.

The advanced form also reports the in-between stages: Checking and Birth are unconfirmed, and Hidden (2) is a stationary target sitting where a moving one already is — using it would count one person twice.

Firmware older than 2.7.0 only has the advanced form, so testing for Valid works on every version; “kw307_set_tracker_state_advanced_mode(true)” makes a newer module match it.

➤ Counter

Roughly “0” – “32”. It rises while the target keeps being detected and falls when it is missed; the slot is released at 0. A high counter is a solid, persistent target; a low one just appeared or is about to drop.

Note: repetitive background clutter — a fan, for instance — never reaches the output: the module reports that slot as Empty, so there is no separate clutter value to test. Tune the rejection with “kw307_set_advanced_clutter_rejection()”.

3. SDK Integration

Only **one include** is required — #include “KW307_api.h”. It pulls in KW307_io.h (lifecycle / frame reception) and KW307_protocol.h (the frame structs), so the whole customer surface is available from that single header. The internal KW307_cmd.h / KW307_uart.h layers need no study; KW307_port.h only matters when porting to a new platform.

3.1 Files

File	Description
KW307_api.h/.c	high-level config API (include this)
KW307_io.h/.c	init / update / receive frames
KW307_protocol.h/.c	frame structs + decode
KW307_cmd.h/.c	internal command encoder
KW307_uart.h/.c	framing + CRC-16
KW307_port.h	the 5 functions you implement per platform

Porting to a new platform = implement the 5 functions in KW307_port.h (copy example_platform/port_template/).

3.2 Quick Start

#include “KW307_api.h”

```
static void on_radar(const KW307Frame_t* f) {
    //f->human_flag, f->moving[0].distance_cm, f->moving[0].angle_tenth ...
}

int main(void) {
    kw307_init();           // opens UART from user_config.h
    kw307_check_fw_version(300); // optional: confirm MCU FW matches

    kw307_set_motion_sensitivity(2, 0); // motion sensitivity level (0..9), byp = 1 for hand_dryer
    kw307_set_stationary_sensitivity(1);
    kw307_set_flag_hold_time(11);

    kw307_on_frame(on_radar);
    for (;;) kw307_update(); // pumps RX, fires the callback
}
```

Every setter returns KW307_OK (0) on success or a negative KW307_ERR_* (see §3.6). Out-of-range arguments return KW307_ERR_PARAM.

3.3 Receiving data — the frame callback

The module streams a data frame about 20 times a second. Instead of making the user constantly poll for new data, the SDK uses a callback: the user supplies a function, and the SDK calls that function every time a frame arrives. This is an event-driven model — the application is notified automatically as each frame is received, so it does not need to poll or block waiting for data.

Step 1 — write the callback. It must have exactly this shape: it takes a pointer to the decoded frame and returns nothing. The frame `f` carries the whole result — the presence flag and all targets (two moving + one stationary), not just one:

```
static void on_radar(const KW307Frame_t* f) {
    bool present = f->human_flag;           // 1 = someone present

    for (int i = 0; i < 2; i++)               // the TWO moving targets
        if (f->moving[i].state == 1) {       // Valid: Simple = 1, Advanced = 3
            int dist = f->moving[i].distance_cm; // cm
            int ang = f->moving[i].angle_tenth / 10; // degrees
        }

    if (f->stationary[0].state == 1) {        // the STATIONARY target; 1 = Valid
        int dist = f->stationary[0].distance_cm;
    }
}
```

The `state == 1` is Valid in the simple form, which is how the module boots. In the advanced form the moving value is 3.

Step 2 — register it, then pump the receiver in the main loop:

```
kw307_init();
kw307_on_frame(on_radar); // hand the user's function to the SDK (no parentheses!)
for (;;) kw307_update();  // each complete frame -> the SDK calls on_radar(&frame)
```

For a GUI, the `f` pointer is only valid during the call, so copy the whole frame out and read it from the user's timer / loop:

```
static KW307Frame_t g_frame; static bool g_have = false;
static void on_radar(const KW307Frame_t* f) { g_frame = *f; g_have = true; }
// UI timer: if (g_have) { g_have = false;
// use g_frame.human_flag / g_frame.moving[0] / moving[1] / stationary[0] }
```

3.4 Human flag

f->human_flag is the module's **built-in presence decision** — a convenient 1 / 0 “someone is here”. Its only tuning knob is kw307_set_flag_hold_time(seconds), which sets how long the flag stays asserted after the last detection (debounce). That setter affects **only** human_flag; it changes nothing else and does not touch the raw tracker outputs.

Treat human_flag as a **default you can override**. The SDK is a general tool: if its built-in presence definition doesn't suit the application, ignore human_flag and derive a custom one from the tracker output — every target's distance, angle and state is in the same frame:

```
/* A custom presence rule (ignore f->human_flag) */
bool present = false;
for (int i = 0; i < 2; i++) /* moving targets */
    if (f->moving[i].state == 1 && /* Valid: Simple = 1, Advanced = 3 */
        f->moving[i].distance_cm <= 300) /* e.g. within 3 m */
        present = true;
if (f->stationary[0].state == 1 && /* stationary Valid */
    f->stationary[0].distance_cm <= 300)
    present = true;
```

On top of this, the application can add a custom hold time, a region limit (distance + field of view), or multi-frame confirmation — whatever the product needs.

3.5 Configuring sensitivity — two ways

A. Level knobs.

Every demo scene in example_platform/ uses these:

```
kw307_set_motion_sensitivity(level, byp_spd); // 0..9 (most->least), byp_spd = 1 for hand_dryer
kw307_set_stationary_sensitivity(level); // 0 (most) .. 9 (least)
```

B. Low-level (Direct thresholds — for fine tuning).

The scenes in example_platform/ use the level knobs, and add the low-level call where a scene needs a specific floor.

```
kw307_set_mag_threshold(moving_floor, still_floor); // detection MAG floor
```

3.6 Return codes

Reference KW307_io.h.

Code	Description
KW307_OK (0)	Success
KW307_ERR_PARAM	argument out of range
KW307_ERR_PORT*	UART open / send / rcv / closed
KW307_ERR_TIMEOUT	no response within timeout
KW307_ERR_NACK	MCU rejected (see kw307_last_nack_status)
KW307_ERR_RESP_LEN / _CRC	bad response length / CRC

API Map (kw307_api.h)

Group	Function	Description
System	kw307_radar_init	Reboot the module.
	kw307_factory_reset	Erase saved settings, restore defaults.
	kw307_save_settings	Save current settings.
	kw307_clear_state	Clear current targets and presence; settings untouched.
	kw307_check_fw_version	Check the module firmware matches this SDK.
Lifecycle / RX (KW307_io.h)	kw307_init	Open UART + SDK initial.
	kw307_close	Close the UART link.
	kw307_is_open	Check if the link is open.
	kw307_on_frame	Register callback for radar frames.
	kw307_update	Pump data every loop; fires the callback.
	kw307_read_blocking	Send a request and wait for the reply.
	kw307_last_nack_status	Get why the last command was rejected.
Output	kw307_set_output_default	Turn on the default output frame.
	kw307_set_output_off	Stop the output stream (radar keeps running).
Configure	kw307_set_power_down	Enter deep sleep.
	kw307_set_gain	Set RF gain.
	kw307_set_detect_range	Set near / far detection distance.
	kw307_set_detect_fov	Set detection width.
	kw307_set_mag_threshold	Set signal floor (higher = less sensitive).
Sensitivity	kw307_set_motion_sensitivity	Set moving-target sensitivity level.
	kw307_set_stationary_sensitivity	Set stationary-target sensitivity level.
Trackers	kw307_set_moving_tracker_cnt	Tune moving-target hold / response time.
	kw307_set_stationary_tracker_cnt	Tune stationary-target hold / response time.
Stationary tuning	kw307_set_force_stationary	Keep presence on for a stationary target within range.
	kw307_set_max_stationary_time	Max time a stationary target is kept.
Clutter rejection	kw307_set_advanced_clutter_rejection	Ignore repetitive motion (fans, AC).
Advanced Mode	kw307_set_tracker_state_advanced_mode	Switch tracker state to the advanced form (Valid = 3, not 1).
Human Flag	kw307_set_flag_hold_time	How long presence stays on after last detection.
	kw307_set_gpio_human_flag	Turn the presence output pin on or off.

4. Function reference — KW307_api.h

Auto-generated from the header doc comments.

➤ System

◆ `kw307_radar_init`

`int kw307_radar_init(void);`

Re-initialise the radar (~130 ms, blocking). Re-applies the current configuration. Call after a setting that needs an init to take effect.

◆ `kw307_factory_reset`

`int kw307_factory_reset(void);`

Clear saved settings and reload firmware defaults.

◆ `kw307_save_settings`

`int kw307_save_settings(void);`

Persist the current settings to flash; restored automatically on next power-up. Don't call in a loop — each call uses a flash write cycle.

◆ `kw307_clear_state`

`int kw307_clear_state(void);`

Drop the targets the module is holding and reset the presence flag, so detection starts over; settings are kept. Targets take a moment to reappear. Needs firmware 2.7.2 or newer.

◆ `kw307_check_fw_version`

`int kw307_check_fw_version(int timeout_ms);`

Check the connected MCU firmware is compatible with this SDK.

Compatible = firmware version $\geq$ the paired baseline (KW307_SDK_FW_MAJOR.MINOR.PATCH).

Compared field-by-field, highest priority first (MAJOR, then MINOR, then PATCH); the first differing field decides and short-circuits, so any newer firmware passes.

0 : compatible

1 : incompatible — major or minor differs; re-verify the protocol

<0 : link error (no response / UART down)

➤ Output mode

◆ `kw307_set_output_default`

`int kw307_set_output_default(void);`

See `kw307_set_output_off` above (shared documentation).

◆ `kw307_set_output_off`

`int kw307_set_output_off(void);`

`kw307_set_output_off()` stop the periodic stream (the radar keeps running); works for whichever frame is active.

`kw307_set_output_default()` (re)select the default frame — presence + 2 moving + 1 still target (consume with `kw307_on_frame`).

➤ Device config

◆ *kw307_set_power_down*

```
int kw307_set_power_down(bool down);
```

Deep-sleep Power Down.

down = true -> RF powered down, lowest current; frame output stops (a UART byte from the host wakes it).

down = false -> Normal (resume sensing).

Boot default: false.

◆ *kw307_set_gain*

```
int kw307_set_gain(uint8_t index);
```

RF gain. index 0~8: -3 / 0 / +3 / +6 / +9 / +12 / +15 / +18 / +21 dB.

Boot default: 5 (+12 dB).

◆ *kw307_set_detect_range*

```
int kw307_set_detect_range(uint16_t min_cm, uint16_t max_cm);
```

Min + max detection distance in cm.

min_cm : near gate (e.g. 50 = 0.5 m). max_cm : far gate (e.g. 1000 = 10 m).

Boot defaults: 50 / 1000. min_cm must be < max_cm.

◆ *kw307_set_mag_threshold*

```
int kw307_set_mag_threshold(int16_t baseline_moving, int16_t baseline_stationary);
```

Mag threshold — detection magnitude floor for moving and stationary targets.

Higher = less sensitive. Range per band 0~32767.

baseline_moving : moving-target floor (default 350)

baseline_stationary : stationary-target floor (default 100)

◆ *kw307_set_detect_fov*

```
int kw307_set_detect_fov(int16_t fov_deg, uint8_t shrink_rate_deg);
```

Detection cone — how wide the radar sees in front.

fov_deg : cone width in whole degrees (1~180). Larger = picks up people off to the sides. Boot default: 120.

shrink_rate_deg : how much the cone narrows per 100cm (0~25 deg/100cm);

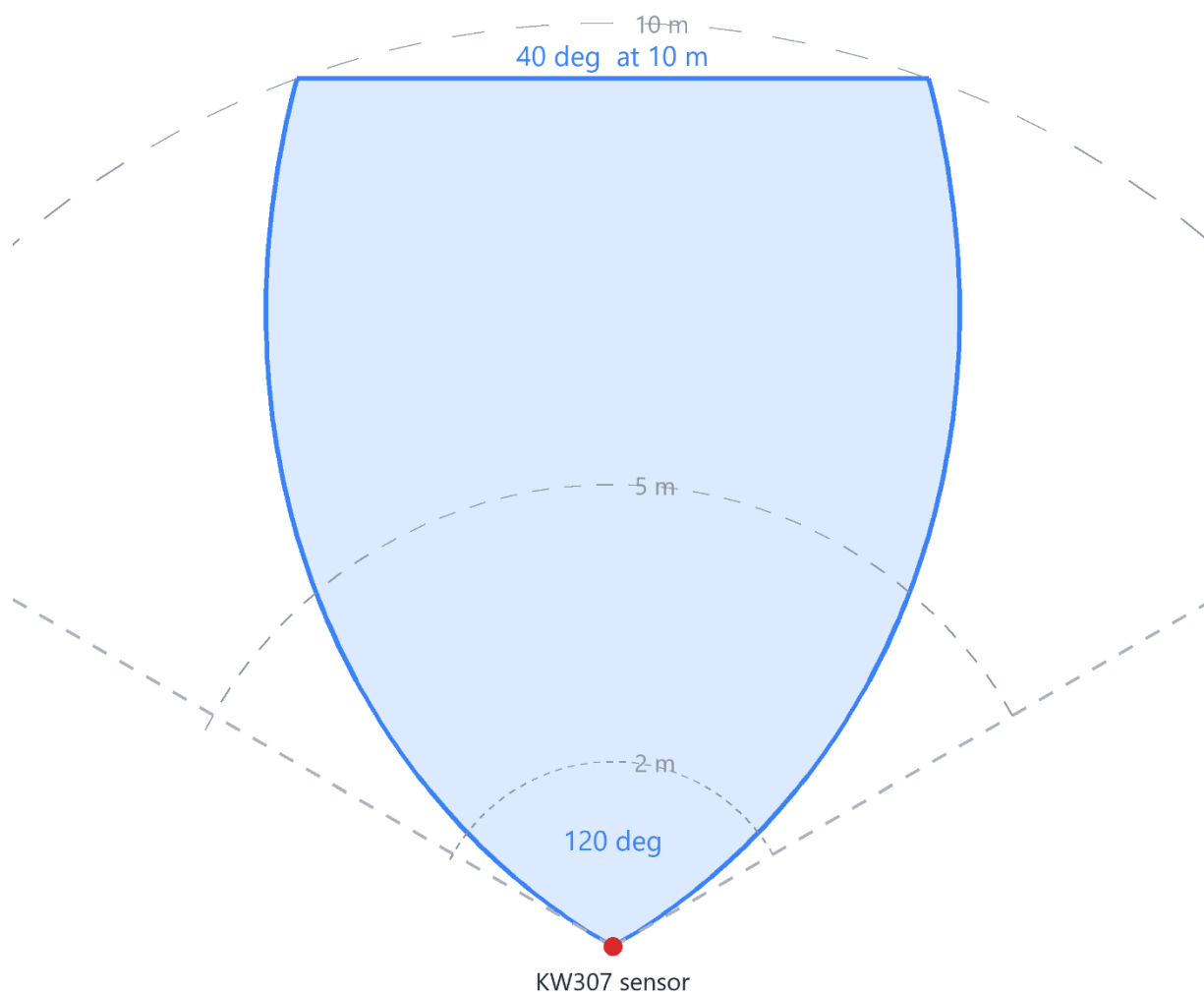
higher = tighter cone at long range. Boot default: 8.

Y This can be used to define a rectangle-like area.

Tuning: false triggers from people passing by -> reduce fov_deg;

missing off-centre targets at long range -> reduce shrink_rate_deg.

Detection FOV: fov = 120 deg, shrink = 8 (deg / m)



The cone is 120 deg at the sensor and narrows 8 deg for every 1 m of range
120 deg @0 m -> 80 deg @5 m -> 40 deg @10 m. Grey dashes = no shrink (stays 120 deg).

◆ *kw307_set_motion_sensitivity*

```
int kw307_set_motion_sensitivity(uint8_t level, uint8_t bypass );
```

Motion (moving-target) sensitivity.

level : 0 (most) -> 9 (least) — a monotonic preset of the moving-band HPF + Fast-band speed gate.

bypass : 0 = normal; non-zero switches the moving HPF to its lowest cutoff freq. and turns the speed-sum gate off, so the tracker reacts to any motion regardless of accumulated speed (e.g. a hand dryer).

Affects the moving band only. level > 9 returns KW307_ERR_PARAM.

◆ *kw307_set_stationary_sensitivity*

```
int kw307_set_stationary_sensitivity(uint8_t level);
```

Stationary (still-target) sensitivity as a single level

0~9 : most-sensitive (0, catches breathing-rate motion) -> least (9);

levels 7~9 also raise the detection floor (+6 dB).

Affects the stationary band only. level > 9 returns KW307_ERR_PARAM.

◆ *kw307_set_moving_tracker_cnt*

```
int kw307_set_moving_tracker_cnt(uint8_t cnt_max, uint8_t cnt_show_th);
```

Tune the moving-target tracker counters.

cnt_max : hold time of a moving target.

how long a target survives without re-detection,
in frames (20 fps). Default 32.

cnt_show_th : response time of a moving target.

The target enters the "Valid" state once cnt >= cnt_show_th.

Default 8. Must be <= cnt_max.

◆ *kw307_set_stationary_tracker_cnt*

```
int kw307_set_stationary_tracker_cnt(uint8_t cnt_max, uint8_t cnt_show_th);
```

Tune the stationary-target tracker counters.

Same meanings as the moving tracker;

Defaults: cnt_max 64, cnt_show_th 8.

◆ *kw307_set_force_stationary*

```
int kw307_set_force_stationary(bool on, int16_t distance_cm, int16_t fov_deg);
```

Keep the presence signal asserted while a still target stands inside a forward window (distance x field-of-view), so brief detection dropouts don't flicker presence while a person sits very still.

on enable (true) / disable (false).

distance_cm window depth in cm (0~3276); a still target must be closer.

fov_deg window width in whole degrees (0~180), edge to edge
(e.g. 60 = +/- 30 deg around directly-ahead).

Boot defaults: on = true, distance_cm = 150, fov_deg = 40.

Tuning: presence flickers while a person sits still -> widen the window;

presence lingers after they leave -> narrow it.

◆ *kw307_set_max_stationary_time*

`int kw307_set_max_stationary_time(uint8_t enable, uint16_t seconds);`

Maximum time a still target may sit before the radar releases it.

enable : 1 = enforce the cap, 0 = never auto-release on time.

seconds : lifetime, 0~3276 s. Higher for offices/lounges, lower for fast-cycle scenes (urinals, hand dryers).

Boot default: enabled, 20 s.

Tuning: presence stays “occupied” too long after the person left -> shorten; person dropped while still sitting -> lengthen, or set enable = 0.

◆ *kw307_set_advanced_clutter_rejection*

`int kw307_set_advanced_clutter_rejection(bool en, int16_t spd_th, int16_t dev_th);`

Reject repetitive background motion (AC, machinery, ...) so it is not reported as presence; Turn it off to report every moving target.

en : true = on (default), false = report every moving target.

spd_th : motion threshold — higher = reject more. Range 0..1000, default 300

dev_th : allowed drift in cm — higher = reject more. Range 0..50, default 20.

◆ *kw307_set_tracker_state_advanced_mode*

`int kw307_set_tracker_state_advanced_mode (bool advanced);`

Tracker-state reporting form.

false : simple (default) — state is 0 Empty / 1 Valid; an Empty slot has its other fields zeroed.

true : advanced — state also reports the in-between stages, and a moving target is Valid at 3 instead of 1. Reporting form only; detection is unchanged. Effective from the next frame.

Firmware older than 2.7.0 always reports the advanced form.

◆ *kw307_set_flag_hold_time*

`int kw307_set_flag_hold_time(uint16_t seconds);`

Presence-flag hold time (debounce): how long the presence flag stays asserted after the last detection, so it doesn't flicker between frames.

seconds : 0~3276 s (held as seconds*20 frames internally); 0 s is floored to 1 frame.

seconds > 3276 returns KW307_ERR_PARAM.

◆ *kw307_set_gpio_human_flag*

`int kw307_set_gpio_human_flag (bool on);`

Presence output pin on / off.

true : the pin follows the presence flag (default).

false : the pin is held low.

Switching it back on applies the presence state the module already holds — it does not restart detection, so a person already in front of the sensor drives the pin high on the next frame. Not saved by kw307_save_settings().

5. Application presets (example scenes)

The scenes are Outdoor, Office, Desk, Urinal and Hand-dryer. Each is simply a bundle of the setters above — a tested starting point to copy and fine-tune. Their values are not repeated here, so they cannot fall out of step with the code that uses them: see DEMO_SCENE_* in example_platform/<port>/main.c. The evaluation GUI applies the same presets from its Application drop-down.

Revision History

Date	Version	Description
2026-08-06	V2.5.1	New APIs; Linux + Raspberry Pi Pico example ports; pairs MCU FW 2.7.5
2026-07-17	V2.4.4	Function Adjustment
2026-07-15	V2.4.3	Doc fix: stationary[].state
2026-07-13	V2.4.2	Update examples.
2026-07-08	V2.4.1	Update Hand Dryer example setting
2026-07-02	V2.3	Official Release
2026-03-23	V1.0	Initial Release

NOTICE & CONFIDENTIALITY STATEMENT

Confidentiality and Proprietary Rights

This document and the information contained herein (including but not limited to register maps, recommended configurations, and functional descriptions) are the proprietary and confidential property of KeywaveTech.

This information is provided to the recipient solely for the purpose of product evaluation and integration.

Any unauthorized use, reproduction, distribution, or disclosure of this document to any third party, in whole or in part, is strictly prohibited without express prior written consent from KeywaveTech.

Confidential